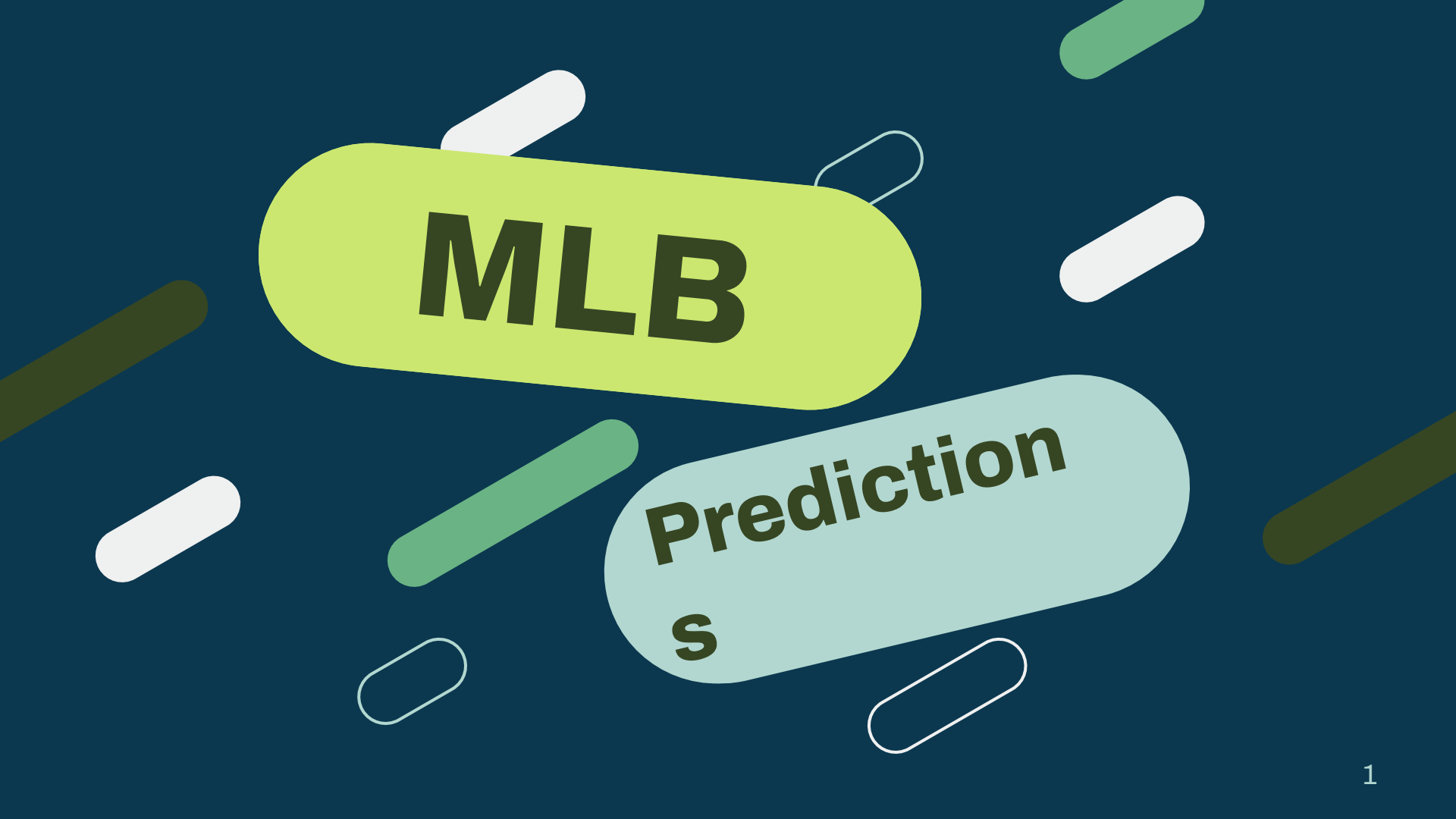


An abstract graphic on a dark blue background. It features two main rounded rectangular shapes: a yellow one at the top containing the text 'MLB' and a light blue one at the bottom containing the text 'Prediction' and a large 'S'. The background is decorated with various geometric elements: several white and green elongated rectangles, two white paperclip-like shapes, and two dark olive green diagonal bars.

MLB

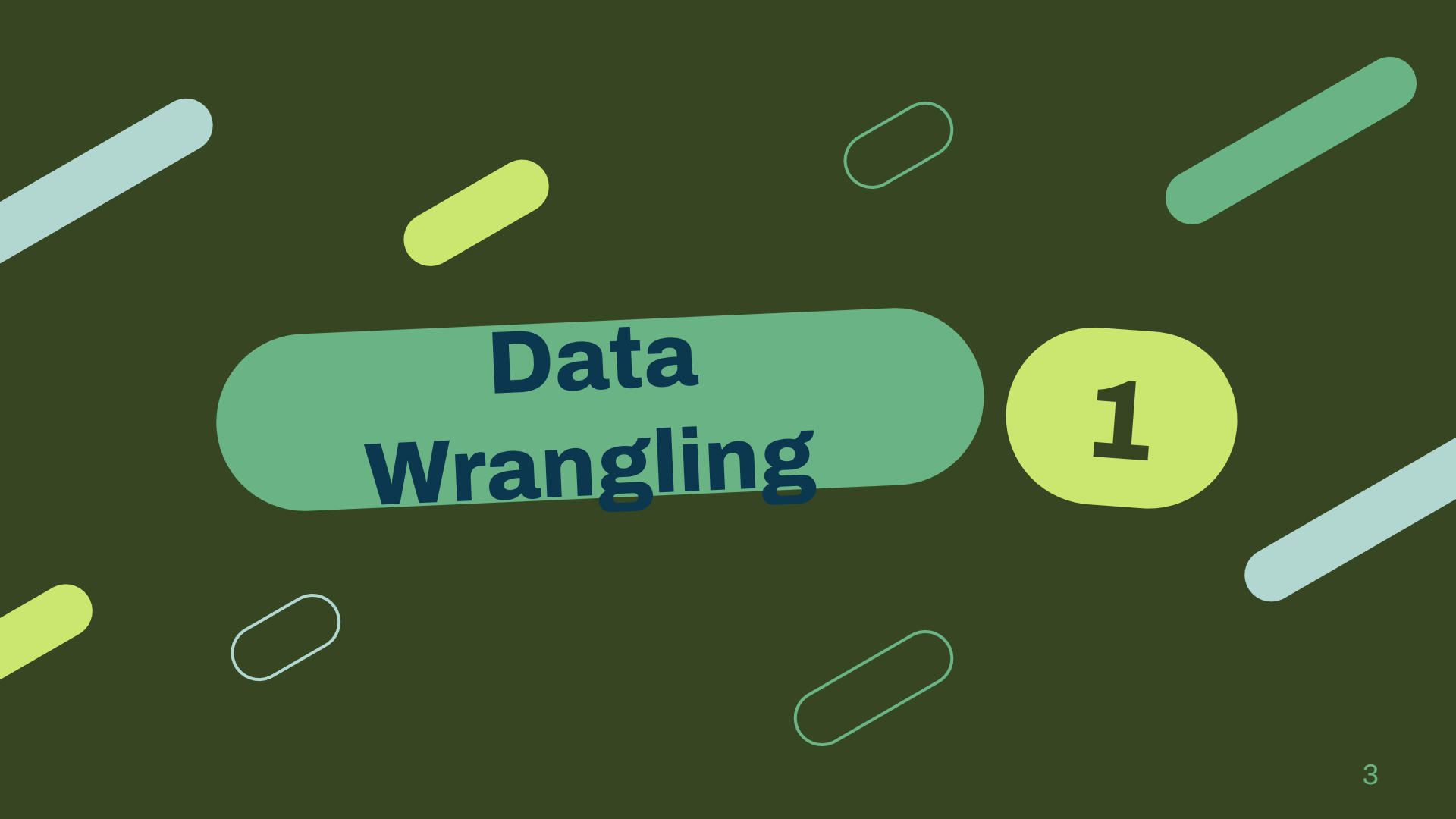
Prediction

S

Introduction

- For our MLB predictions project we have chose to **predict the winner** of a single MLB game based on each team's statistics before the game.
- We will dive in depth about acquiring data, calculating stats for each team, and developing a **logistic regression model**.
- This model uses each teams data to determine whether or not the home team will win (1) or lose (0).



The background is a solid dark olive green. It is decorated with several geometric shapes: light blue bars, yellow bars, and teal bars, all oriented diagonally. There are also white-outlined pill shapes scattered across the background. The main title is centered in a teal pill shape, and the number '1' is in a yellow circle to its right.

Data Wrangling

1

RetroSheet

- Utilizing retrosheet.org we were able to download every game log from **2000-2024**
- Using Python and pandas, we loaded in each text file and concatenated the DataFrames for each year, creating a single comprehensive dataset with game logs from all years

Initial Data

59204 rows, 162 columns

Each game/row consisted of the following fields:

Date & Game Info – Game date, number (single/double/triple-header), and day of the week

Teams & Scores – Home & visiting teams, leagues, and final scores

Game Length – Measured in outs (54 for full game) and total minutes played

Attendance – Number of spectators

Performance Stats

- **Offense** – Hits, home runs, RBIs, strikeouts, stolen bases
- **Pitching** – Earned runs, wild pitches, balks
- **Defense** – Errors, putouts, double plays

Officials & Key Players – Umpires, managers, winning/losing pitchers

Add New Fields

Summary of New Fields Created

1. **Calculate Singles (1B) for Each Team:**
 - Derived by subtracting extra-base hits (doubles, triples, homers) from total hits.
2. **Runs Allowed:**
 - Visiting team's runs allowed = Home team's runs.
 - Home team's runs allowed = Visiting team's runs.
3. **Innings Pitched:**
 - Computed using the `calc_IP` (next slide) function on each team's linescore.
4. **Pitching Stats:**
 - Hits & walks given by each team are assigned from the opponent's hitting stats.
5. **Run Differential and Game Outcome:**
 - `run_diff`: Difference between home and visiting team runs.
 - `home_win`: 1 if home team won, 0 otherwise.

Calculating Innings Pitched

Function: `calc_IP(linescore)`

1. **Remove 'x'** from `linescore` (home team may not bat in the last inning).
2. **Extract innings scores** using regular expressions:
 - Find total runs scored every inning, including those in parentheses (e.g., `(13)`).
 - Store them in a list.
3. **Count total innings** from the list length.
4. **Return the count** as total innings pitched.

Example:

Input: `'0021000(13)0'`

Output: 9

Input: `'00210030x'` (home team doesn't hit in the 9th) -> (visiting team doesn't pitch)

Output: 8

Important Statistics

- In order to successfully predict the winner of a game we need more comprehensive team statistics that give an overview of each team's performance prior to every game
- Our goal is to calculate a 30 day rolling average for each team's statistics in addition to their seasonal averages

Offensive and Defensive:

AVG:

- H / AB

SLG:

- $(1B) + (2 \times 2B) + (3 \times 3B) + (4 \times HR) / AB$

OBP:

- $H + BB + HBP / AB + BB + HBP + SF$

OPS:

- $SLG + OBP$

ERA:

- $ER \times 9 / IP$

WHIP:

- $BB + H / IP$

Extra:

Pythagorean Exception:

- Estimates the probability that Team 1 will defeat Team 2 based on their win percentages.
- $(\text{runs scored})^2 / [(\text{runs scored})^2 + (\text{runs allowed})^2]$

Log5:

- Analyzes Team 1's and Team 2's scoring performance to assess matchup dynamics.
- $P(j \text{ beats } k) = (P_{jj} - (P_{jj} \cdot P_{kk})) / (P_{jj} + P_{kk} - (2 \cdot P_{jk} \cdot P_{kk}))$ where P_{jj} is the percentage of games won by team j and P_{kk} is the percentage of games won by team k

Aggregating Stats

Team Statistics Overview:

Create a Team Dictionary:

- Store team names as keys
- Team-specific DataFrames are values

Build Team-Specific DataFrames:

- For each team, extract all games where they played as either the home or away team.

Calculate Team Seasonal and 30 Day Performance Metrics:

- Hitting Stats: OPS, SLG, OBP, AVG
- Pitching Stats: WHIP, ERA
- Win Percentage
- Total Runs Scored Vs Runs Allowed

Merge with Master DataFrame:

- Integrate team results with the main dataset based on the corresponding game dates.



Feature Engineering

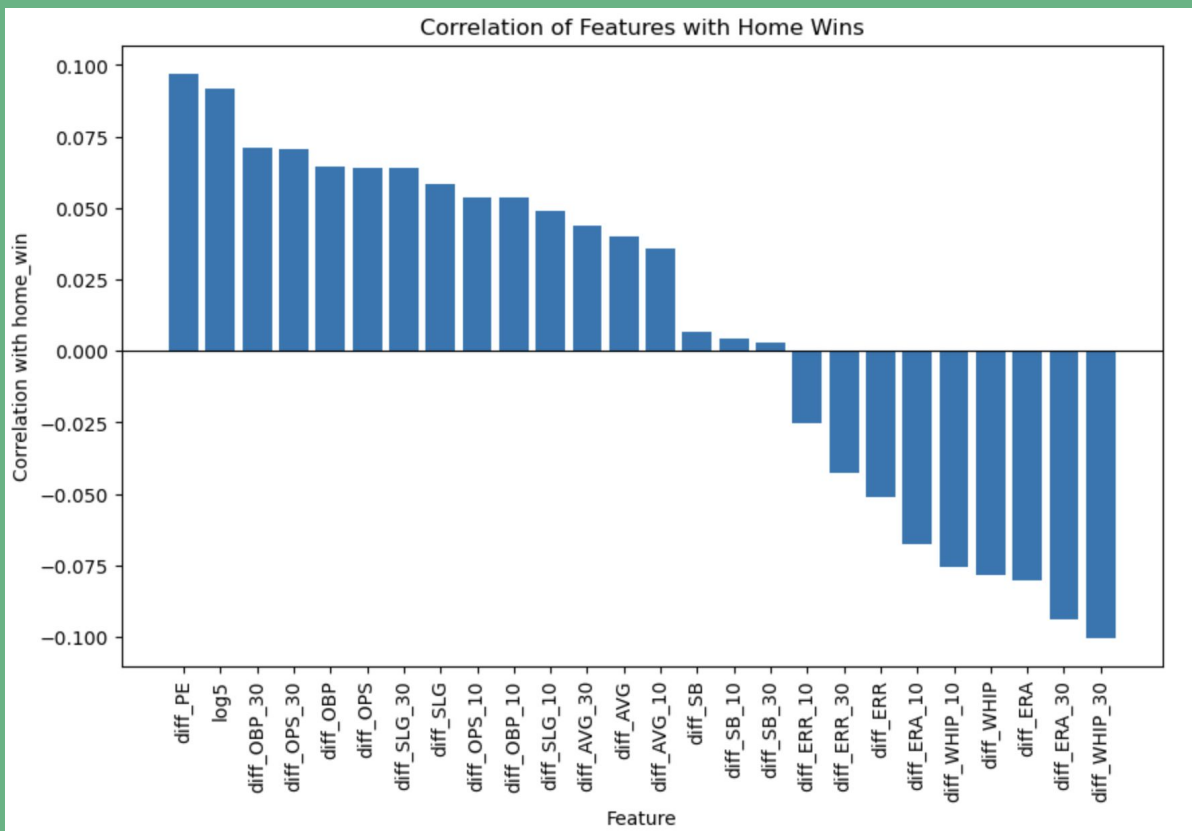
2

Differences

Calculated difference between home team
and visiting team's relevant stats
SLG, ERA, WHIP, etc...

Correlations

Find most relevant features.



Correlations

The seasonal statistics and 30 day statistics for OBP, SLG, ERA, and WHIP have the most correlation with 'home_win' so we chose to focus on these features

- diff_PE
- log5
- diff_OBP
- diff_SLG
- diff_OBP_30
- diff_SLG_30
- diff_ERA
- diff_WHIP
- diff_ERA_30
- diff_WHIP_30

Selecting Features

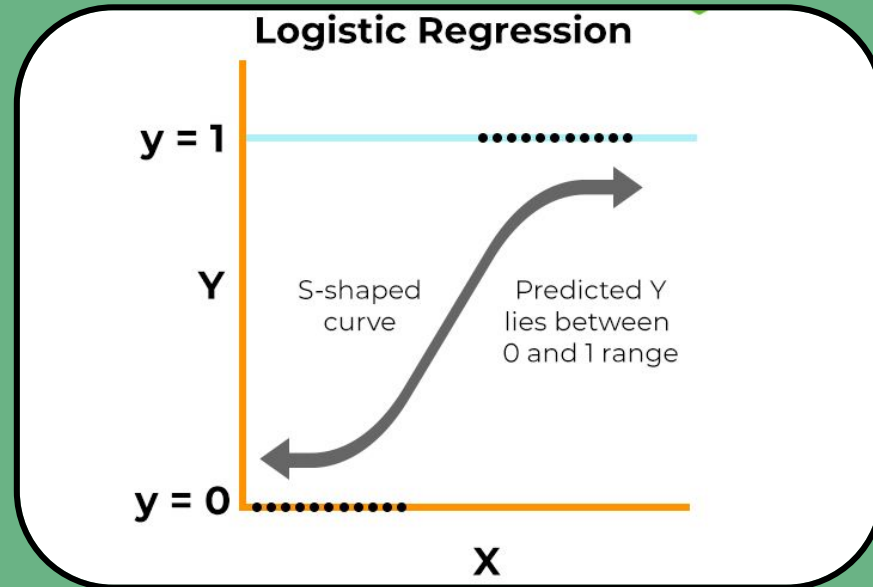


Logistic Regression Model

3

Training

Split data into training, validation, and test sets



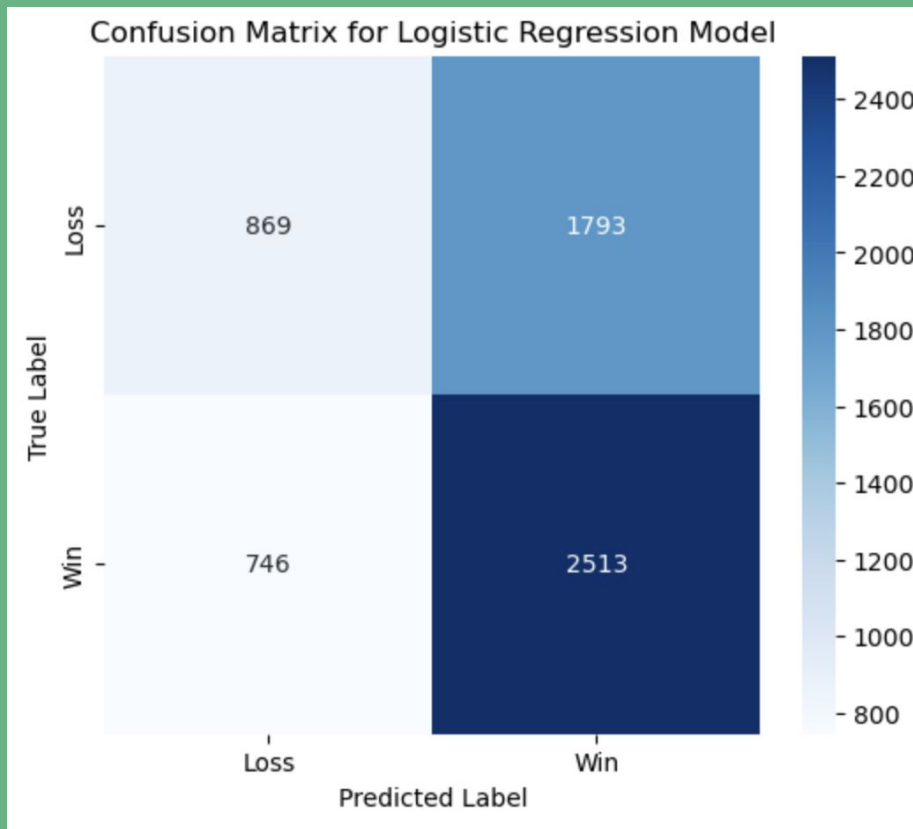
Results

Train Log Loss: 0.6829

Validation Log Loss: 0.6849

Test Accuracy: 0.5712

Results



From the testing data our model has never seen, it predicted the following outcomes. A lot of the model's errors come from false positives.

Next Steps

To increase model accuracy, we might find starting pitcher data.

The background is a solid teal color. Scattered across the surface are several pill-shaped elements. Some are solid colors: dark olive green, light green, and off-white. Others are white outlines of pill shapes. These elements are oriented diagonally in various directions, creating a dynamic, abstract pattern.

Thank You!